

《优化理论与算法》第 11 次作业

姓名：焦传斌 学号：2024111223

- 作业截止于周五（5/22/2026），在 Canvas 系统中提交 PDF 或照片文件
- 作业题目的 tex 文件可以在 Canvas 中下载
- 鼓励相互讨论，但不要抄袭.

2.1 计算 $P_C\left(\begin{bmatrix} 1 \\ 2 \end{bmatrix}\right)$ ，其中 $C = \{x \in \mathbb{R}^2 | x_1^2 + 4x_2^2 \leq 4\}$ 是个椭圆。

解：记待投影点为 $a = (1, 2)^T$ 。因为

$$1^2 + 4 \cdot 2^2 = 17 > 4,$$

所以投影点在椭圆边界上。考虑

$$\min_x \frac{1}{2} \|x - a\|_2^2, \quad \text{s.t. } x_1^2 + 4x_2^2 \leq 4.$$

取拉格朗日函数

$$L(x, \lambda) = \frac{1}{2}((x_1 - 1)^2 + (x_2 - 2)^2) + \frac{\lambda}{2}(x_1^2 + 4x_2^2 - 4), \quad \lambda \geq 0.$$

KKT 条件给出

$$x_1 - 1 + \lambda x_1 = 0, \quad x_2 - 2 + 4\lambda x_2 = 0.$$

因此

$$x_1 = \frac{1}{1 + \lambda}, \quad x_2 = \frac{2}{1 + 4\lambda}.$$

再由边界条件 $x_1^2 + 4x_2^2 = 4$ 得

$$\frac{1}{(1 + \lambda)^2} + \frac{16}{(1 + 4\lambda)^2} = 4.$$

该方程在 $\lambda \geq 0$ 上的根为

$$\lambda \approx 0.292234.$$

故

$$P_C(a) = \begin{bmatrix} \frac{1}{1 + \lambda} \\ \frac{2}{1 + 4\lambda} \end{bmatrix} \approx \begin{bmatrix} 0.773853 \\ 0.922110 \end{bmatrix}.$$

代回可验算 $0.773853^2 + 4(0.922110)^2 \approx 4$ 。

2.2 给定二阶锥 $C = \{(x, t) \in \mathbb{R}^n \times \mathbb{R} \mid \|x\|_2 \leq t\}$, 试证

$$P_C(v, s) = \begin{cases} 0 & \|v\|_2 \leq -s \\ (v, s) & \|v\|_2 \leq s \\ \frac{1}{2}(1 + \frac{s}{\|v\|_2})(v, \|v\|_2) & \|v\|_2 \geq |s| \end{cases}$$

提升: 你可以从如下式子开始, 并使用 KKT 定理。

$$\begin{aligned} \min_{x, t} \quad & \frac{1}{2}\|x - v\|_2^2 + \frac{1}{2}\|t - s\|_2^2 \\ \text{s.t.} \quad & \|x\|_2 \leq t \end{aligned}$$

证明: 令 $r = \|v\|_2$ 。若 $(v, s) \in C$, 即 $r \leq s$, 则投影点就是自身,

$$P_C(v, s) = (v, s).$$

下面考虑 $r > s$ 。锥约束要求 $t \geq 0$ 。固定一个 $t \geq 0$ 时, 内层问题为

$$\min_{\|x\|_2 \leq t} \frac{1}{2}\|x - v\|_2^2,$$

也就是把 v 投影到半径为 t 的欧式球上。因此

$$x(t) = \begin{cases} v, & r \leq t, \\ \frac{t}{r}v, & 0 \leq t < r. \end{cases}$$

于是原问题化为一维问题

$$\min_{t \geq 0} \frac{1}{2}(t - s)^2 + \frac{1}{2}(\max\{r - t, 0\})^2.$$

当 $t \geq r$ 时, 目标为 $\frac{1}{2}(t - s)^2$ 。若 $r \leq s$, 最优解为 $t = s$, 这就是第二种情况。

当 $0 \leq t \leq r$ 时, 目标为

$$\phi(t) = \frac{1}{2}(t - s)^2 + \frac{1}{2}(r - t)^2.$$

对 t 求导得

$$\phi'(t) = (t - s) + (t - r) = 2t - r - s.$$

若 $r + s \geq 0$, 则驻点为

$$t^* = \frac{r + s}{2}.$$

在条件 $r \geq |s|$ 下, 有 $0 \leq t^* \leq r$, 所以

$$x^* = \frac{t^*}{r}v = \frac{1}{2}\left(1 + \frac{s}{r}\right)v, \quad t^* = \frac{1}{2}(r + s).$$

因此

$$P_C(v, s) = \frac{1}{2}\left(1 + \frac{s}{\|v\|_2}\right)(v, \|v\|_2), \quad \|v\|_2 \geq |s|.$$

最后, 若 $r + s \leq 0$, 即 $r \leq -s$, 则 $\phi(t)$ 在 $t \geq 0$ 上从 $t = 0$ 开始递增, 最优 $t^* = 0$, 从而 $x^* = 0$ 。故

$$P_C(v, s) = 0, \quad \|v\|_2 \leq -s.$$

三种情况合并即得题中公式。

2.3 给定 $f(x) = \|x\|_2^2$, 计算 $\text{prox}_{tf}(x)$ 。

解: 接近端算子的定义,

$$\text{prox}_{tf}(x) = \arg \min_u \left\{ \frac{1}{2} \|u - x\|_2^2 + t \|u\|_2^2 \right\}.$$

目标函数可微且强凸, 对 u 求梯度并令其为零:

$$(u - x) + 2tu = 0.$$

所以

$$(1 + 2t)u = x, \quad \text{prox}_{tf}(x) = \frac{1}{1 + 2t}x.$$

2.4 考虑如下函数

$$f(x) = \frac{1}{2}(x_1 + 2x_2 - 3)^2 - \sum_{i=1}^2 \log(x_i).$$

假设起始点 $x_0 = (10, 10)$, 步长为 $t_0 = 1$ 。

- 展示一次梯度下降法迭代
- 展示一次近端梯度下降法迭代 ($h(x) = -\sum_{i=1}^2 \log(x_i)$)

解: 将函数写成

$$f(x) = g(x) + h(x), \quad g(x) = \frac{1}{2}(x_1 + 2x_2 - 3)^2, \quad h(x) = -\sum_{i=1}^2 \log x_i.$$

在 $x_0 = (10, 10)$ 处,

$$x_{0,1} + 2x_{0,2} - 3 = 10 + 20 - 3 = 27.$$

因此

$$\nabla g(x_0) = (27, 54)^T, \quad \nabla h(x_0) = \left(-\frac{1}{10}, -\frac{1}{10}\right)^T.$$

一次普通梯度下降使用完整梯度

$$\nabla f(x_0) = \left(27 - \frac{1}{10}, 54 - \frac{1}{10}\right)^T = (26.9, 53.9)^T.$$

步长 $t_0 = 1$ 时,

$$x_1^{\text{GD}} = x_0 - \nabla f(x_0) = (10, 10)^T - (26.9, 53.9)^T = (-16.9, -43.9)^T.$$

这个点不在 $\log$ 函数的定义域内, 说明直接梯度下降会越出可行区域。

一次近端梯度下降先对光滑部分 g 做梯度步:

$$y = x_0 - \nabla g(x_0) = (10, 10)^T - (27, 54)^T = (-17, -44)^T.$$

然后计算

$$x_1^{\text{PG}} = \text{prox}_h(y).$$

对每个分量,

$$\text{prox}_{-\log}(y_i) = \arg \min_{u_i > 0} \left\{ \frac{1}{2}(u_i - y_i)^2 - \log u_i \right\}.$$

一阶条件为

$$u_i - y_i - \frac{1}{u_i} = 0, \quad u_i^2 - y_i u_i - 1 = 0.$$

取正根得

$$u_i = \frac{y_i + \sqrt{y_i^2 + 4}}{2}.$$

所以

$$x_1^{\text{PG}} = \left(\frac{-17 + \sqrt{293}}{2}, \frac{-44 + \sqrt{1940}}{2} \right)^T \approx (0.058621, 0.022716)^T.$$

近端步给出的点仍满足两个分量为正。

2.5 考虑如下函数

$$f(x) = \frac{1}{2}x^T A x + b^T x + \lambda \|x\|_1$$

其中, b 为从 $[-1, 1]^n$ 中随机生成的向量, A 为随机凸对称矩阵。 A 的构造方法考虑令 $A = B^T B + D$, 其中 B 为随机稀疏矩阵, D 为非负对角矩阵用来控制 A 矩阵条件数。你可以自己选择合适的 n 及其他生成参数。

编程实现下列方法:

- 次梯度下降法
- 近端梯度法
- 加速近端梯度法 (FISTA)
- 下降版本的 FISTA 方法
- 重启版本的 FISTA 方法

选取三个不同的 λ 绘制三张图, 每幅图中绘制目标函数值与迭代次数的关系

解: 实验中取 $n = 80$, 随机生成稀疏矩阵 $B \in \mathbb{R}^{100 \times 80}$, 非零密度为 0.08, 并令

$$A = B^T B + D, \quad D = \text{diag}(0.05, \dots, 1.00).$$

这样 A 为对称半正定矩阵; 由于 D 的对角元为正, 实际生成的 A 为正定矩阵。向量 b 从 $[-1, 1]^n$ 上均匀随机生成, 初始点取 $x^0 = 0$ 。

记

$$g(x) = \frac{1}{2}x^T A x + b^T x, \quad h(x) = \lambda \|x\|_1.$$

光滑部分梯度为

$$\nabla g(x) = Ax + b,$$

其 Lipschitz 常数取

$$L = \lambda_{\max}(A).$$

近端类方法使用步长 $t = 1/L$ 。 ℓ_1 项的近端算子是软阈值算子：

$$\text{prox}_{t\lambda\|\cdot\|_1}(z)_i = \text{sign}(z_i) \max\{|z_i| - t\lambda, 0\}.$$

五种方法的实现如下。

$$\text{次梯度下降: } x^{k+1} = x^k - \alpha_k (Ax^k + b + \lambda \text{sign}(x^k)), \quad \alpha_k = \frac{0.8}{L\sqrt{k+1}}.$$

$$\text{近端梯度法: } x^{k+1} = \text{prox}_{t\lambda\|\cdot\|_1}(x^k - t(Ax^k + b)).$$

FISTA 在预测点 y^k 上做同样的近端梯度步，并加入动量。下降版本 FISTA 若候选点 u 满足

$$f(u) \leq f(x^k),$$

则接受 u ，否则保留 x^k 。重启版本 FISTA 使用函数值重启准则：若新点使目标函数上升，则把动量参数重置并从当前点重新做一次近端梯度步。

下面分别取

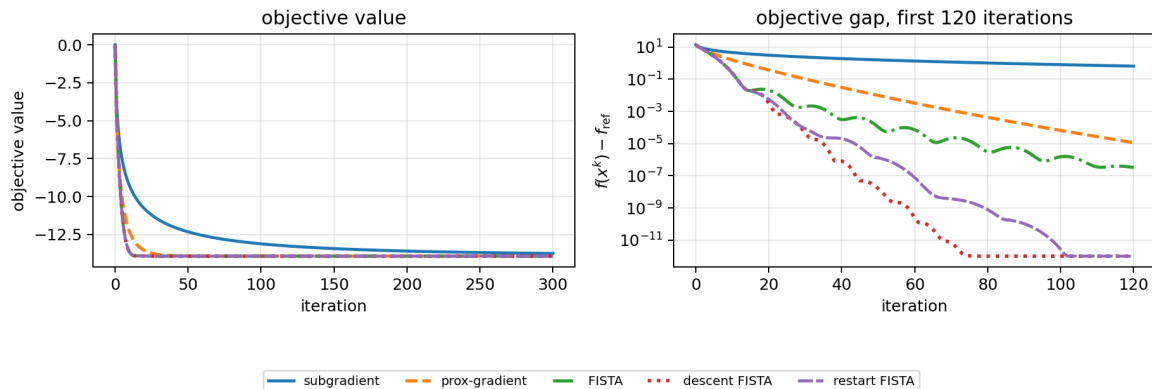
$$\lambda \in \{0.02, 0.10, 0.50\},$$

每种方法迭代 300 步。完整代码见附录。为了更清楚地区分不同方法，每张图左侧给出目标函数值，右侧给出前 120 次迭代的半对数目标函数差值

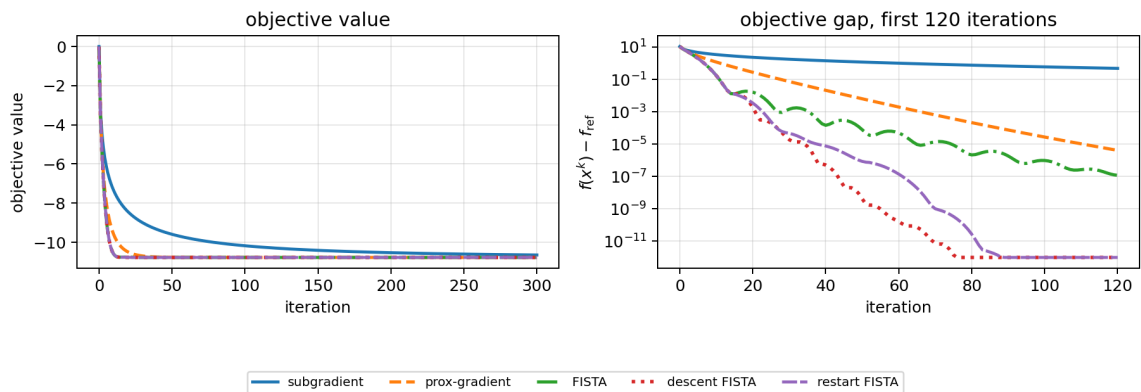
$$f(x^k) - f_{\text{ref}},$$

其中 f_{ref} 取本次实验所有方法达到的最小目标函数值。

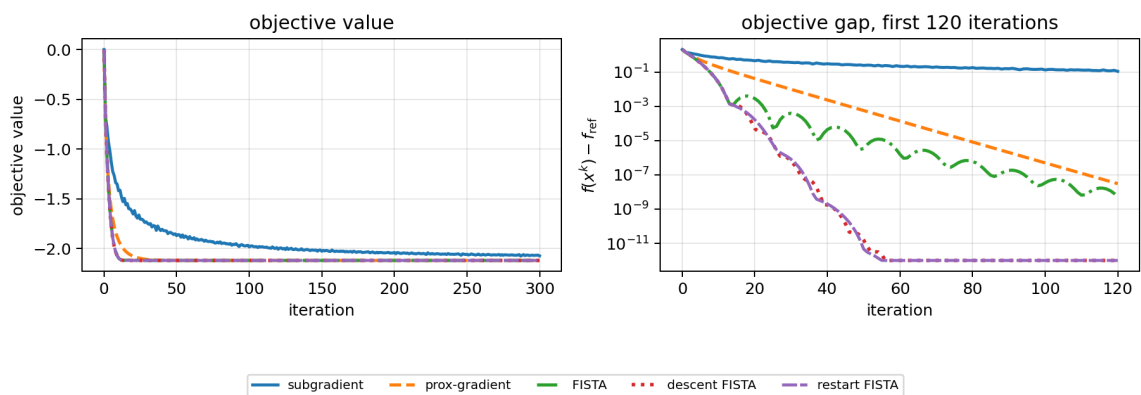
lambda = 0.02



lambda = 0.10



lambda = 0.50



从图中可以看到，次梯度法下降最慢；近端梯度法稳定下降；FISTA 类方法前期下降更快。右侧半对数图能更明显看出加速方法和普通近端梯度法之间的收敛速度差异。

A 完整代码

第 2.5 题使用的完整 Python 代码如下。

```

1 from pathlib import Path
2
3 import matplotlib
4
5 matplotlib.use("Agg")
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9
10
11 BASE_DIR = Path(__file__).resolve().parent
12 FIG_DIR = BASE_DIR / "figures"

```

```

13 FIG_DIR.mkdir(exist_ok=True)
14
15
16 def build_problem(seed=20260514):
17     rng = np.random.default_rng(seed)
18     n = 80
19     m = 100
20     density = 0.08
21
22     mask = rng.random((m, n)) < density
23     B = rng.normal(size=(m, n)) * mask / np.sqrt(m * density)
24     D = np.diag(np.linspace(0.05, 1.0, n))
25     A = B.T @ B + D
26     b = rng.uniform(-1.0, 1.0, n)
27     L = float(np.linalg.eigvalsh(A).max())
28     return A, b, L
29
30
31 def objective(A, b, lam, x):
32     return 0.5 * float(x @ A @ x) + float(b @ x) + lam * float(np.linalg.norm(x
33     , 1))
34
35 def grad(A, b, x):
36     return A @ x + b
37
38
39 def soft_threshold(z, tau):
40     return np.sign(z) * np.maximum(np.abs(z) - tau, 0.0)
41
42
43 def subgradient_descent(A, b, L, lam, iterations):
44     x = np.zeros_like(b)
45     values = []
46     for k in range(iterations + 1):
47         values.append(objective(A, b, lam, x))
48         if k == iterations:
49             break
50         alpha = 0.8 / (L * np.sqrt(k + 1.0))
51         subgrad = grad(A, b, x) + lam * np.sign(x)

```

```

52     x = x - alpha * subgrad
53     return np.array(values)
54
55
56 def proximal_gradient(A, b, L, lam, iterations):
57     x = np.zeros_like(b)
58     step = 1.0 / L
59     values = []
60     for k in range(iterations + 1):
61         values.append(objective(A, b, lam, x))
62         if k == iterations:
63             break
64         x = soft_threshold(x - step * grad(A, b, x), step * lam)
65     return np.array(values)
66
67
68 def fista(A, b, L, lam, iterations):
69     x = np.zeros_like(b)
70     y = x.copy()
71     q = 1.0
72     step = 1.0 / L
73     values = []
74     for k in range(iterations + 1):
75         values.append(objective(A, b, lam, x))
76         if k == iterations:
77             break
78         x_next = soft_threshold(y - step * grad(A, b, y), step * lam)
79         q_next = 0.5 * (1.0 + np.sqrt(1.0 + 4.0 * q * q))
80         y = x_next + ((q - 1.0) / q_next) * (x_next - x)
81         x = x_next
82         q = q_next
83     return np.array(values)
84
85
86 def descent_fista(A, b, L, lam, iterations):
87     x = np.zeros_like(b)
88     y = x.copy()
89     q = 1.0
90     step = 1.0 / L
91     values = []

```



```

92     for k in range(iterations + 1):
93         values.append(objective(A, b, lam, x))
94         if k == iterations:
95             break
96         candidate = soft_threshold(y - step * grad(A, b, y), step * lam)
97         if objective(A, b, lam, candidate) <= objective(A, b, lam, x):
98             x_next = candidate
99         else:
100             x_next = x.copy()
101             q_next = 0.5 * (1.0 + np.sqrt(1.0 + 4.0 * q * q))
102             y = x_next + ((q - 1.0) / q_next) * (x_next - x)
103             x = x_next
104             q = q_next
105     return np.array(values)
106
107
108 def restart_fista(A, b, L, lam, iterations):
109     x = np.zeros_like(b)
110     y = x.copy()
111     q = 1.0
112     step = 1.0 / L
113     values = []
114     for k in range(iterations + 1):
115         values.append(objective(A, b, lam, x))
116         if k == iterations:
117             break
118         x_next = soft_threshold(y - step * grad(A, b, y), step * lam)
119         if objective(A, b, lam, x_next) > objective(A, b, lam, x):
120             q = 1.0
121             y = x.copy()
122             x_next = soft_threshold(y - step * grad(A, b, y), step * lam)
123             q_next = 0.5 * (1.0 + np.sqrt(1.0 + 4.0 * q * q))
124             y = x_next + ((q - 1.0) / q_next) * (x_next - x)
125             x = x_next
126             q = q_next
127     return np.array(values)
128
129
130 def plot_for_lambda(A, b, L, lam, iterations=300):
131     methods = [

```

```

132     ("subgradient", subgradient_descent(A, b, L, lam, iterations)),
133     ("prox-gradient", proximal_gradient(A, b, L, lam, iterations)),
134     ("FISTA", fista(A, b, L, lam, iterations)),
135     ("descent FISTA", descent_fista(A, b, L, lam, iterations)),
136     ("restart FISTA", restart_fista(A, b, L, lam, iterations)),
137 ]
138
139 k = np.arange(iterations + 1)
140 styles = {
141     "subgradient": dict(color="#1f77b4", linestyle="-", linewidth=1.9),
142     "prox-gradient": dict(color="#ff7f0e", linestyle="--", linewidth=2.0),
143     "FISTA": dict(color="#2ca02c", linestyle="-. ", linewidth=2.0),
144     "descent FISTA": dict(color="#d62728", linestyle=":", linewidth=2.2),
145     "restart FISTA": dict(color="#9467bd", linestyle=(0, (5, 1)), linewidth
146 =1.9),
147 }
148
149 f_ref = min(float(np.min(values)) for _, values in methods)
150 gap_floor = 1e-12
151
152 fig, axes = plt.subplots(1, 2, figsize=(10.2, 4.2), dpi=180)
153 for label, values in methods:
154     axes[0].plot(k, values, label=label, **styles[label])
155 axes[0].set_xlabel("iteration")
156 axes[0].set_ylabel("objective value")
157 axes[0].set_title("objective value")
158 axes[0].grid(True, alpha=0.3)
159
160 zoom_end = min(120, iterations)
161 zoom = k <= zoom_end
162 for label, values in methods:
163     gaps = np.maximum(values - f_ref, gap_floor)
164     axes[1].semilogy(k[zoom], gaps[zoom], label=label, **styles[label])
165 axes[1].set_xlabel("iteration")
166 axes[1].set_ylabel(r" $f(x^k) - f_{\text{ref}}$ ")
167 axes[1].set_title(f"objective gap, first {zoom_end} iterations")
168 axes[1].grid(True, which="both", alpha=0.3)
169
170 handles, labels = axes[0].get_legend_handles_labels()
171 fig.legend(handles, labels, loc="lower center", ncol=5, fontsize=8)

```

```

171     fig.suptitle(f"lambda = {lam:.2f}", y=0.98)
172     fig.tight_layout(rect=(0.0, 0.14, 1.0, 0.92))
173     file_name = f"hw11_lambda_{lam:.2f}".replace(".", "_") + ".png"
174     fig.savefig(FIG_DIR / file_name)
175     plt.close()
176
177     print(f"lambda={lam:.2f}")
178     for label, values in methods:
179         print(f"    {label:14s}: final objective = {values[-1]:.6f}")
180
181
182 def main():
183     A, b, L = build_problem()
184     print(f"L = {L:.6f}")
185     for lam in (0.02, 0.10, 0.50):
186         plot_for_lambda(A, b, L, lam)
187
188
189 if __name__ == "__main__":
190     main()

```